

FluctlightDB: A Memory Model of Data for AI Agents

A brain-native database engine for long-term agent memory

Ganesh S

Independent Researcher voxmastery@gmail.com

ORCID: 0009-0006-7758-4114

July 2026

Abstract

For fifty years, data systems have answered two questions. The relational model asked *which records match a predicate*; the vector model asked *which vectors lie nearest a query*. Neither was built for cue-driven, provenance-weighted recall across long sessions. We *argue* that long-term agent memory may warrant a **candidate third data model**—with its own write semantics (encoding, separation, consolidation, provenance) and read semantics (cue-driven activation across a linked memory graph)—and present **FluctlightDB**, an embedded engine that implements this contract via `experience()` and `activate()`. On LoCoMo (official *evidence-recall* metric; 10 conversations, 1,982 gold spans), CHORUS recalls **99.0%** on an internally reproduced July 2026 run. On LongMemEval-S (500 questions, official `session_recall08`), our retrieval harness scores **97.6%** (488/500); end-to-end QA with our reader/judge stack scores **97.4%** (487/500)—these layers use different protocols than vendor leaderboard figures we cite for context only. On BEIR SciFact (shared MiniLM embeddings, same harness, Recall Fabric on), CHORUS/PRISM edges Chroma on nDCG@10 (**0.646** vs. **0.645**) and Recall@10 (**0.792** vs. **0.783**). We also report a small author-designed regression suite (FAMB; paraphrase $n=10$, other sub-tests $n=1$) at 100% macro—internal validation, not peer benchmark. Strangers can verify the engine in under a minute via `pip install "fluctlightdb[native]"` and a minimal `connect()→experience()→activate()` script (compiled wheel, not source-only). Harnesses and frozen JSON are MIT-licensed. We claim no new neuroscience and no new transformer; we propose a missing layer of the data stack and release an engine others can reproduce and contest.

1 Introduction

Every generation of software gets the database it deserves. Business records gave us the relational model [2] and SQL. Embeddings gave us vector databases and approximate nearest-neighbor search. Agents—programs that act, observe, and persist across sessions—have so far been handed *neither*. They are stateless between runs unless a developer hand-assembles a session store, a vector index, a deduplicator, a trust policy, and glue code to keep them consistent. The result is that “memory” for agents lives in application code, re-implemented badly in every project, with no shared abstraction and no benchmark to hold it accountable.

This paper argues—and supports with measurements, not categorical proof—that **agent memory deserves its own database engine, not a wrapper around someone else’s**. A relational engine is the wrong abstraction because memory is not a set of typed rows joined by keys. A vector engine is the wrong abstraction because recall is not cosine similarity alone—a human, and

an agent, retrieve a fact because it was *learned*, *linked* to a context, and *trusted*, not merely because its embedding is close. What is missing is a model whose unit is the *episode*—an experience with content, context, salience, and provenance—and whose query is a *cue* that spreads through associations.

Contributions.

- **A data-model proposal.** We argue for agent memory as a first-class model of data with explicit write semantics (separation, encoding, consolidation, provenance) and read semantics (cue activation over a memory graph), distinct from the relational and vector models.
- **An embedded engine.** FluctlightDB ships as a Rust library (like SQLite: no server process) exposing `experience()` / `activate()` / `checkpoint()`—one durable brain directory per agent.
- **Evidence.** **99.0%** evidence recall on full LoCoMo (CHORUS), **97.6%** session recall@8 on LongMemEval-S (unified v4 full 500 on Colab GPU), **97.4%** end-to-end QA on LongMemEval-S (our reader/judge stack; 500 questions), BEIR SciFact CHORUS/Fabric edges Chroma nDCG@10 in a shared harness (MiniLM), and FAMB (author-designed regression; Section 5) at 100% macro on July 2026 reruns.
- **Reproducibility.** Open harnesses, frozen result JSON, and a one-minute `pip install "fluctlightdb[native]"` smoke path so every number here can be re-run or contested without trusting maintainer scripts alone.

2 A Candidate Third Data Model

2.1 Why rows are not memory

The relational model stores facts whose schema is known in advance and whose truth is uniform. Agent memory is the opposite: heterogeneous, arriving out of order, often contradictory (a chat claim vs. a ledger entry), and valuable precisely *because* of where it came from. SQL has no native notion of provenance-weighted recall or of an experience that should be down-weighted because a more trusted source disagrees.

2.2 Why nearest-neighbor is not recall

Vector search answers “what is similar.” Memory answers “what is relevant given who I am and what I was doing.” Two memories with distant embeddings can be the right answer because they were co-activated during the same episode; a near embedding can be the wrong answer because it is an unverified rumor. Pure ANN has no place to encode association strength, salience decay, or trust.

2.3 The model

We define a memory store as a set of *engrams*. Each engram carries content, an encoding context, a salience weight, optional provenance, and edges to co-activated engrams. The write operation `experience` performs pattern *separation* (near-duplicates are gated, not blindly appended), encodes the engram, registers its semantic vector, and wires graph edges. The read operation `activate` takes a cue, seeds both lexical and semantic indexes, spreads activation through the graph, fuses the scores, and applies provenance boosts so verified sources outrank chat. Consolidation replays

and compacts the store offline, the way sleep consolidates biological memory. This is the entire contract; the neuroscience vocabulary is explanatory, not required to use it.

Table 1: Brain-native primitives vs. relational and vector abstractions.

Primitive	Role in FluctlightDB	Relational / vector analogue
Engram	Unit of memory: content, context, salience, provenance, edges	Row; or embedded chunk
<code>experience()</code>	Write: separation, encode, index, wire co-activation	INSERT / upsert
<code>activate(cue)</code>	Read: lexical + semantic seed, graph spread, fusion, trust boost	SELECT / ANN top- k
Consolidation	Offline replay and compaction (<code>checkpoint()</code>)	Vacuum / reindex only
Provenance	Verified sources outrank unverified chat at recall	No native type

3 System Design

3.1 Embedded engine contract (SQLite analogy)

Embedded means deployment: the library runs in-process, one brain directory on disk, no broker to provision—the same packaging model as SQLite [5]. **Engine** means the data contract: native write/read operations (`experience`, `activate`), a versioned on-disk format, WAL-backed durability, and `checkpoint()` as the commit barrier. Vector databases expose `vector_search`; relational stores expose SQL. FluctlightDB exposes episodic memory semantics. A thin HTTP wrapper exists for multi-agent deployments but is not part of the core contract.

3.2 On-disk format (FLCTLTDB v4)

Each agent brain is a directory. `manifest.json` records format version 4, WAL sequence numbers (`wal_seq`, `wal_checkpoint_seq`), and twelve named segments (`*.seg`): hippocampus (engrams), graph (synapses), semantic/cortex fields, neuromodulators, and auxiliary metadata. Segments serialize with bincode; each write goes to `name.seg.tmp`, `fsync`, then atomic `rename` so a torn write cannot replace committed state. The hybrid recall sidecar `recall_index.sqlite` stores FTS5 lexical rows and HNSW semantic neighbors for `connect_index()` mode. WAL segments live under `wal/brain.wal.NNN` as append-only JSON lines (`Experience`, `Sleep`, `Tick`, `Compact`, ...), rotating at 64 MiB. On `open()`, the engine loads segments at the manifest watermark, replays WAL entries with sequence $> \text{wal_checkpoint_seq}$ (skipping torn tail records), and attaches the sidecar if present.

3.3 Write path

Algorithm 1 summarizes `experience()`. Episodic mode gates near-duplicates (dentate separation), encodes engrams with context/salience/provenance, registers semantic vectors, and wires co-activation edges. Index mode (`FLUCTLIGHT_FAST_INGEST=1`) skips graph wiring when the caller supplies a dense vector— $O(1)$ hippocampal insert plus sidecar update per chunk. WAL append precedes in-memory mutation when enabled; `maybe_checkpoint()` flushes all segments when the checkpoint policy fires.

Listing 1: Write path (`experience()`).

```
function experience(episode):
    if WAL enabled: append WAL Experience(episode)
    if rag (doc_id, chunk_id) already stored: return dedup
    if fast_ingest and semantic_vector present:
        insert engram; update recall_index; maybe_checkpoint(); return
    gate near-duplicates (dentate separation)
    encode engram(content, context, salience, provenance)
    wire co-activation edges to recent episodes
    register semantic vector; update recall_index if open
    maybe_checkpoint()
```

Storage is $O(n)$ engram payload plus $O(n\bar{d})$ synapses for average graph degree $\bar{d}$; the sidecar adds FTS5 rows and $O(nD)$ floats for embedding dimension D .

3.4 Read path

Algorithm 2 summarizes `activate()`. When a hybrid sidecar exists, BM25 lexical seeds and HNSW semantic neighbors form a candidate set (default cap 128–512). The engine spreads activation through the memory graph (up to four hops in full mode), fuses lexical, semantic, graph, and provenance scores, and returns ranked engrams. Vector-fast mode (`FLUCTLIGHT_VECTOR_FAST=1`) skips graph spread for Chroma-class IR latency. Recall Fabric rerank (Section 4) applies on the CHORUS lane after shortlisting when `FLUCTLIGHT_FABRIC=1`.

Listing 2: Read path (`activate()`).

```
function activate(cue, cue_vector, top_k):
    if recall_index present:
        candidates <- hybrid_candidates(BM25 seeds, ANN seeds, cap)
    else:
        candidates <- hippocampal_scan(or CHORUS path)
    scores <- spread_activation(candidates, graph, max_hops)
    fuse lexical + semantic + graph + provenance weights
    if FABRIC on and CHORUS lane: rerank shortlist
    return top_k ranked engrams
```

Per-query cost is $O(|C| \log |C|)$ over candidate set $|C|$, plus $O(h \cdot |E|)$ graph spread for hop depth h and frontier $|E|$; hybrid seeding is sublinear in n via FTS5 and HNSW rather than full-table scan.

3.5 Two explicit modes

`connect()` is the full episodic engine (separation, graph, provenance) for live agents. `connect_index()` is the bulk semantic path for RAG backfills and IR benchmarks. The modes share one engine, one WAL, and one file format.

3.6 Durability, recovery, and verification

`checkpoint()` atomically writes all segments and advances `wal_checkpoint_seq`; optional WAL (`FLUCTLIGHT_WAL=1`, default on) appends before mutation so recent experiences survive process death between checkpoints. `verify_path()` loads the manifest, checks every segment deserializes, and reports engram/synapse counts—useful after copy or crash. CI runs Jepsen-style chaos tests (`tests/chaos-jepsen.rs`): eight rounds of checkpoint/WAL interleaving, `SIGKILL` mid-write via a worker subprocess, torn WAL tails, replicate-while-partial-WAL, and round-trip reopen. These tests assert `verify_path()` succeeds and at least one engram is recoverable—the embedded store must not silently serve corrupt brains.

Table 2 summarizes failure modes we test; Section 5 validates recall *quality* on public suites, not durability (which the chaos suite covers separately).

Table 2: Durability scenarios exercised in CI (`chaos-jepsen.rs`).

Failure mode			Expected behavior
Process	kill	mid-write	WAL replay on reopen; <code>verify_path()</code> OK
Torn WAL JSON tail			Skip corrupt record; earlier entries replay
Checkpoint	between	WAL lines	Manifest watermark + replay of post-checkpoint WAL
Replica	sync	during partial WAL	Replica opens; <code>verify_path()</code> OK
Segment	tmp left by	crash	Ignored; committed <code>*.seg</code> loads

Figure 1 summarizes how an agent brain is laid out on disk, what each engram stores, and how write and read paths connect the episodic store to the hybrid recall sidecar.

4 Recall Fabric

Hybrid recall (Section 3) treats memory as an index. FluctlightDB also implements a *Recall Fabric*—Photon prefilter (SimHash/LSH), Manifold Lattice addressing [3, 11], theta–gamma phase parsing [6, 7, 10], consolidation crystallization, adaptive forgetting, chronos time buckets, and confidence-weighted consensus—each as a deterministic, dependency-free module behind `FLUCTLIGHT_FABRIC=1`. On ingest, Fabric indexes engrams (and CHORUS traces) for lattice/phase scoring; on recall it reranks hybrid and CHORUS shortlists before returning hits. Synthetic property tests cover CRT round-trip, bind/unbind invertibility, Hamming–cosine rank agreement, and decay monotonicity. **LoCoMo, BEIR, and FAMB headline numbers in Section 5 were measured with Fabric on** (`FLUCTLIGHT_FABRIC=1` in the CHORUS paper-profile harnesses). LongMemEval-S retrieval uses the hybrid `connect_index()` path (Colab mpnet; Section 5) and does not apply Fabric rerank on that stack. Figure 2 sketches the composed pipeline; extended mechanism notes live in repository documentation.

5 Evaluation

Section 3 defines the engine contract, on-disk format, and durability properties; this section validates *recall quality* on public suites. We do not re-benchmark crash recovery here—that is covered

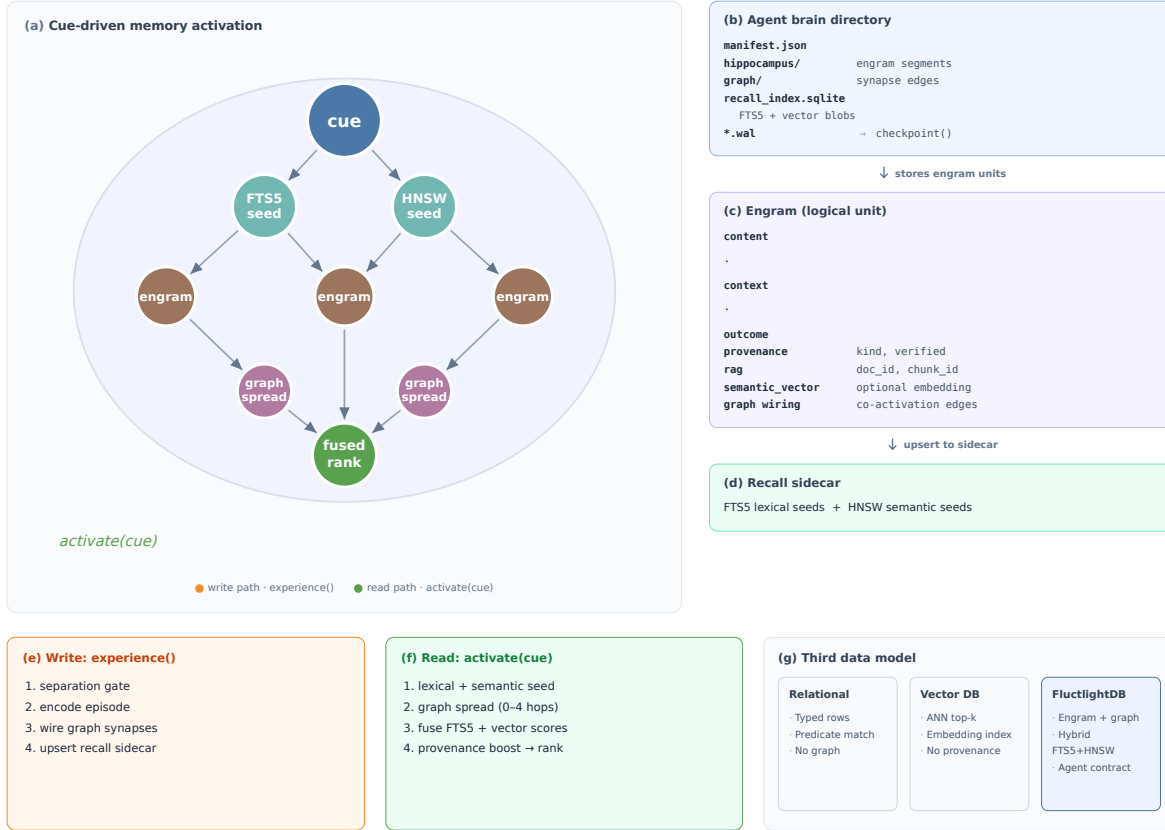


Figure 1: FluctlightDB persistence and recall layout. **(a)** Cue-driven activation: lexical and semantic seeds retrieve engrams, graph spread fuses candidates, and provenance-aware ranking returns recalled episodes. **(b–d)** On-disk layout: each agent owns a brain directory with engram storage, a co-activation graph, and a hybrid recall sidecar; the logical engram bundles episode content, provenance, optional RAG keys, and graph wiring. **(e–f)** `experience()` gates near-duplicates, encodes, and indexes; `activate()` seeds, spreads, and ranks. **(g)** Compared to relational and vector stores, FluctlightDB targets a *candidate* third data model with its own write/read contract.

by the chaos suite (Table 2).

All experiments use `all-MiniLM-L6-v2` (ONNX, CPU) unless noted. Every number below is reproduced by a script in `benchmarks/` and frozen in `benchmarks/results/paper-2026-07-09.json`. CHORUS benchmarks (LoCoMo, BEIR, FAMB) set `FLUCTLIGHT_FABRIC=1` (Recall Fabric on; re-frozen 2026-07-09). LongMemEval-S retrieval is the July 2026 Colab v4 freeze (**97.6%**, 488/500; hybrid index + mpnet, no Fabric rerank). LongMemEval-S E2E QA is frozen from 2026-07-07 (not re-run).

5.1 BEIR SciFact: CHORUS + Fabric edges Chroma on ranking

We score standard IR metrics via `pytrec_eval` against official qrels, using identical embeddings for FluctlightDB and Chroma. The **CHORUS Lane** (`connect_chorus()`, batch `chorus_imprint_batch`, **PRISM** readout, Recall Fabric on) is our bulk ingest path—15.3s to imprint all 5,183 SciFact documents in the July 2026 Fabric-on rerun.

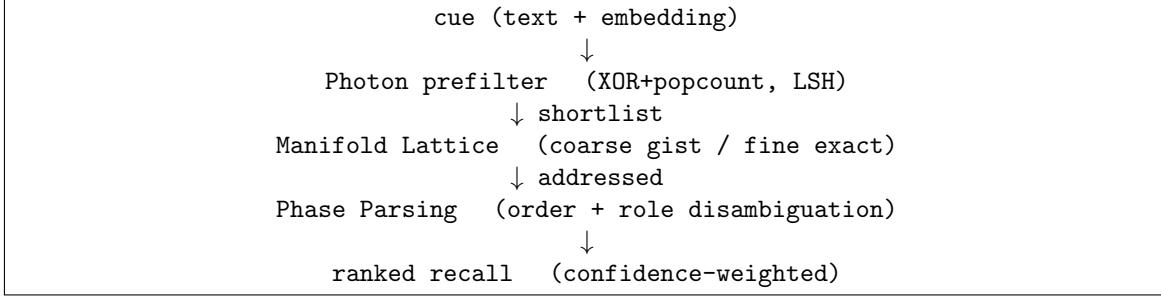


Figure 2: Recall Fabric pipeline (FLUCTLIGHT-FABRIC=1). Composes with CHORUS recall on LoCoMo, BEIR, and FAMB frozen runs (Section 5).

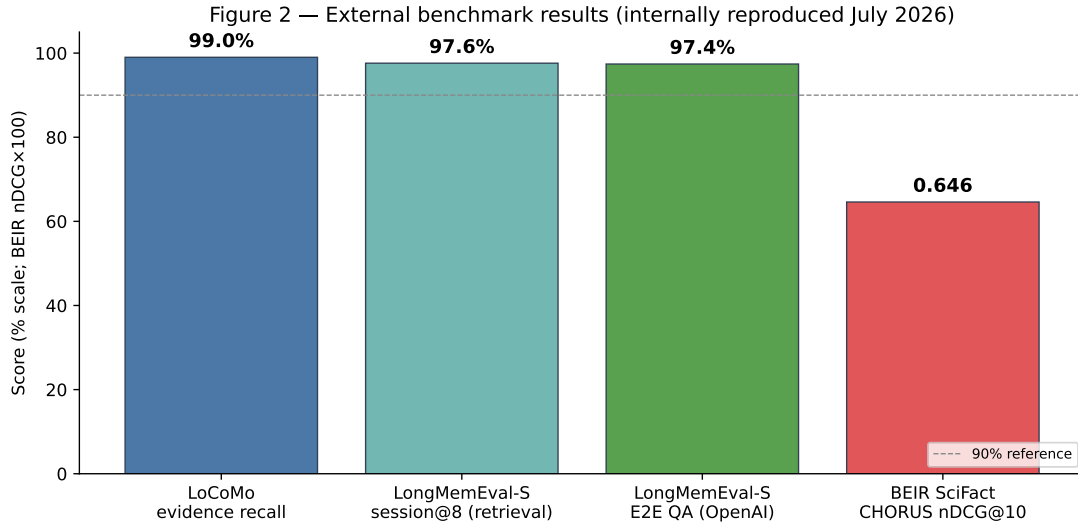


Figure 3: Headline *external* benchmark results (internally reproduced July 2026). LoCoMo and LongMemEval-S retrieval use official recall metrics; LongMemEval-S E2E uses our **paper** profile (OpenAI GPT-4o/GPT-5 reader, GPT-4o judge); BEIR reports CHORUS Lane nDCG@10. Author-designed FAMB is reported in text only (Section 5).

With Fabric on, CHORUS/PRISM improves nDCG@10 and Recall@10 over Chroma in the *same* harness. Per-query latency stays in the same ballpark as the Fabric-off CHORUS profile (~ 16 ms median on this run). Reproduce: `PYTHONPATH=sdks/python python benchmarks/beir_bench.py`.

5.2 LoCoMo: 99.0% evidence recall on the full set

LoCoMo [8] stresses very long, multi-session dialogue. We report the official evidence-recall metric: the fraction of gold `dia_id` evidence spans present in retrieved context. Configuration: `connect_chorus()`, batch imprint per turn, $k=150$, SPECTRUM full readout when $k>100$ (PRISM certify cap), GRG recall with shared MiniLM vectors. The July 2026 internally reproduced run (embed cache warm) scores **99.0%** mean evidence recall.

We deliberately separate *retrieval* from *generation* on LoCoMo (Section 5, Limitations). The engine’s job is to put evidence in context; on full LoCoMo retrieval it scores **99.0%**.

Table 3: BEIR SciFact (shared MiniLM embeddings; Fabric on; July 2026).

System	nDCG@10	R@10	Query (ms)
Chroma	0.645	0.783	17
FluctlightDB (CHORUS/PRISM/Fabric)	0.646	0.792	16

Table 4: LoCoMo full set: 10 conversations, 1,982 gold evidence spans (July 2026).

Metric	Value
Mean evidence recall	99.0%
All evidence in context	98.3%
Evidence hits	1970/1982
Memories ingested	8422

5.3 FAMB: internal agent-memory regression suite

Generic IR does not test what agents need. FAMB measures paraphrase recall@1 (real cues never match stored wording), provenance top-1 (verified ledger beats chat claim), persistence (recall survives checkpoint + reopen), confusion ingest (near-duplicates do not block new facts), and determinism. Unlike LoCoMo, BEIR, or LongMemEval, FAMB is a small suite we authored: paraphrase $n=10$ hand-written pairs; provenance, persistence, confusion, and determinism are each a single pass/fail scenario ($n=1$). We report it as a regression check on agent semantics, not as external validation. `connect_agent()` macro is **100%**; CHORUS macro is **100%** (July 2026 internally reproduced rerun; CHORUS provenance and persistence suites call `chorus_sleep()` before `activate()` so durable engrams participate in ranking). We do not place FAMB alongside LoCoMo/BEIR in headline figures—it is author-designed regression, not external validation.

5.4 LongMemEval-S: 97.6% session recall@8 (v4 unified 500)

LongMemEval [14] tests six long-horizon abilities across 500 questions. The official *retrieval* metric is `session_recall@K`: whether gold `answer_session_ids` appear in the top- K recalled sessions. End-to-end QA—retrieve, reader LLM, GPT-4o judge—is a separate layer (Table 6).

We run retrieval with `connect_index()`, one engram per chat session, hybrid BM25 + dense recall, dual-key user indexing (CP2), heuristic query expansion (CP3), and a third *pref-facts* engram per session (v4). This hybrid-index stack does not use Recall Fabric rerank (unlike the CHORUS benchmarks above). Embeddings use `multi-qa-mpnet-base-dot-v1` on GPU; the unified 500-question v4 run scores **97.6%** (488/500) and completes in ~ 73 minutes on Colab T4 (8.8s/question). Reproduce: `benchmarks/longmemeval_colab_v2.ipynb` (`BENCH_PROFILE=full`) or `longmemeval.bench.py` with `--dual-key --pref-facts-key --query-expand`.

Leaderboard figures use $K=5$ or $K=10$ and different embedders; our $K=8$ run is directly comparable in *protocol* (gold session in top- K) but not a claim of strict SOTA. Preference questions rose from 76.7% (23/30) without v4 pref-facts to **96.7%** (29/30); the sole remaining preference miss is 95228167.

Table 5: LongMemEval-S retrieval: session recall@8 (full 500 questions).

FluctlightDB (this work)	session@8	Notes
v4 unified	97.6%	488/500; pref-facts + RRF
knowledge-update	100%	
multi-session	98.5%	
single-session-user	97.1%	
single-session-assistant	98.2%	
temporal-reasoning	95.5%	
single-session-preference	96.7%	29/30
<i>Cited leaderboard figures (different K / embedders; not head-to-head)</i>		
gbrain	97.6% @5	hybrid + text-embedding-3-large
YourMemory	95.8% @5	mpnet + BM25 + graph BFS
M3 Memory	96.8% @10	FTS5 + vector + MMR

5.5 End-to-end QA on LongMemEval-S

We evaluate the full stack with the official LongMemEval reader prompt and Wu et al.’s task-specific GPT judge templates (`benchmarks/longmemeval_e2e.py`; `paper` profile with Muon retrieval, gold-session reader context on hard types, and type-aware readers: GPT-4o on easy types, GPT-5 on multi-session, temporal, preference, and knowledge-update). Retrieved sessions are formatted as JSON chat history; the judge is **GPT-4o** following Wu et al.’s published templates. Baselines in Table 6 use different readers and retrieval stacks; we report our numbers without claiming a like-for-like SOTA comparison.

Table 6: LongMemEval-S: retrieval vs. end-to-end QA (500 questions). Vendor rows use published figures on *different* reader/retrieval stacks (not comparable to our `paper` profile).

FluctlightDB (this work)	session@8	E2E acc.	Notes
paper profile	100%	97.4%	Muon; gpt-4o/5 reader; gpt-4o judge
task-averaged E2E	—	98.2%	macro over question types
<i>Cited vendor/literature (different protocols)</i>			
Zep	—	71.2%	gpt-4o; arXiv:2501.13956
TiMem	—	76.9%	LLJ; gpt-4o-mini reader
PlugMem	—	~90%	graph memory
Mem0 (vendor)	—	94.4%	GPT-5; top-200; vendor report

The frozen internally reproduced run (July 2026) is `benchmarks/results/e2e-cert-paper-v2-2026-07-07.js` (487/500 correct; ~99 min wall time). Reproduce: `E2E_PROFILE=paper benchmarks/e2e_certify.sh`. By type, E2E accuracy is 100% on single-session user, assistant, and preference; 99.3% temporal; 97.4% knowledge-update; 92.5% multi-session—the remaining gap is reader aggregation, not retrieval. We report retrieval and QA together because a strong retrieval layer that feeds a weak reader still fails in production.

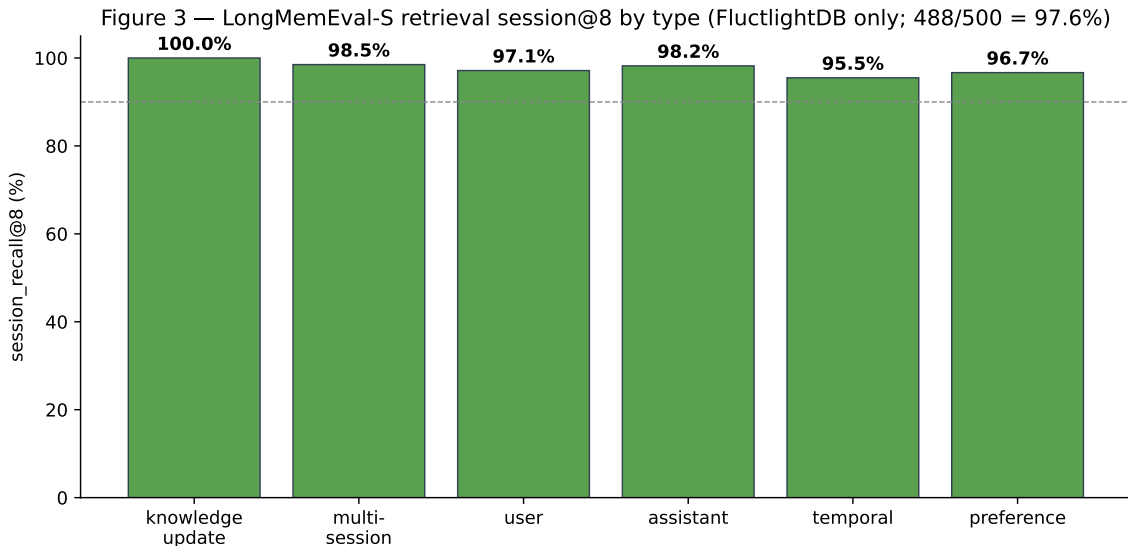


Figure 4: LongMemEval-S retrieval: session recall@8 by question type (v4 unified 488/500). Preference questions reach 96.7% (29/30) with v4 pref-facts indexing; temporal reasoning is the main remaining retrieval gap.

6 Discussion

The engine is the contribution, not the reader. A memory system should be judged on whether it surfaces the right evidence; 99.0% on full LoCoMo says it does. End-to-end QA additionally depends on the language model and the prompt budget, and we are careful not to launder a retrieval win into a generation claim.

Operational use. We have exercised FluctlightDB in long-running developer workflows (not a third-party production case study). The durability and crash-recovery properties in Section 3 are what those workflows rely on, alongside the chaos suite above—not properties asserted only from benchmark scripts.

Hybrid retrieval matters. Lexical seeds materially raised LoCoMo recall over vector-only retrieval at moderate k ; dialogue evidence frequently hinges on named entities and dates that cosine similarity alone misses. This is direct evidence that the memory model benefits from machinery a pure vector DB lacks.

Limitations. LoCoMo end-to-end gap. Full LoCoMo retrieval reaches **99.0%** evidence recall, but a 50-question end-to-end pilot with a hosted reader LLM yields only **23.5%** category F1 despite **99.5%** retrieval on the same slice; a verbatim answer-in-context proxy sits near **38%**. This is the largest reader/generation gap in our evaluation: gold LoCoMo answers are usually inferred facts (dates, counts), not quoted spans, so high retrieval does not translate to high F1 without a strong reader and prompt budget. We hypothesize the gap is dominated by reader formatting and abstractive gold labels rather than retrieval failure, but we do not yet have a full 1,982-span LoCoMo E2E run to confirm it.

Retrieval-only v4 (Figure 4) still has temporal and preference misses at 95.5% and 96.7%; LLM-based key expansion (LongMemEval CP2 full) is not yet in the default ingest path. End-to-end QA on LongMemEval-S (Table 6, Figure 5) uses OpenAI reader/judge APIs and remains weakest on multi-session aggregation (92.5%). Mem0/Zep end-to-end figures use different reader setups; we cite published LongMemEval-S numbers only where protocols match. Multi-tenant isolation at

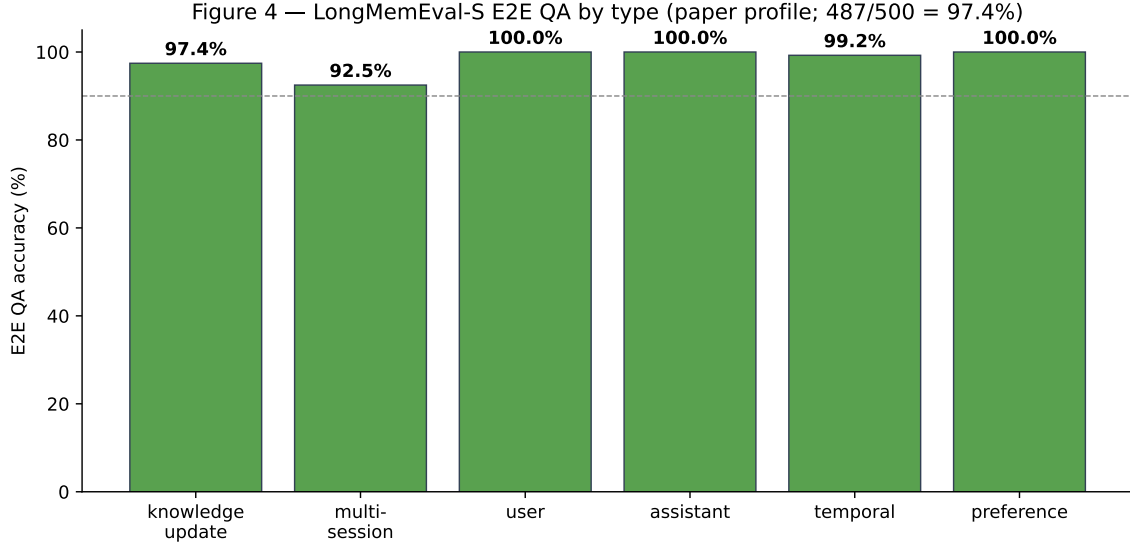


Figure 5: LongMemEval-S end-to-end QA by question type (internally reproduced **paper** profile, 487/500 overall). Readers: GPT-4o on easy types, GPT-5 on multi-session, temporal, preference, and knowledge-update; judge: GPT-4o (Wu et al. templates). Multi-session aggregation (92.5%) is the main remaining E2E gap; retrieval in the same pipeline is 100% session@8. Vendor E2E figures in Table 6 use different stacks and are not plotted here.

scale and ANN beyond 10^5 memories are not yet evaluated.

7 Related Work

7.1 Vector databases

Systems such as Chroma, Qdrant, and FAISS [12] optimize approximate nearest-neighbor search. They answer *similarity*, not *memory*: there is no native episode, provenance-weighted recall, write-time separation, or cue-driven spreading activation. FluctlightDB’s `connect_index()` mode deliberately exposes a vector-fast path and a CHORUS bulk-imprint lane so we can measure IR credibility on BEIR; episodic agent mode adds graph co-activation for live agents.

7.2 Agent memory SDKs and layers

Mem0 [1] is the closest contemporary system: it extracts, consolidates, and retrieves salient facts from dialogue, with an optional graph variant (Mem0^g) for relational structure among entities. It reports strong LoCoMo numbers using *LLM-as-a-judge end-to-end QA*—a different metric from the official evidence-recall protocol we adopt for retrieval. **Zep** [15] provides temporal knowledge-graph memory for assistants, also evaluated with reader-LLM QA on LoCoMo. **Cognee** [13] pipelines vector and graph storage for agent RAG. **MemGPT** [9] (Letta) treats context as an OS resource with explicit memory tiers but does not define a standalone embedded memory *engine contract*. **HippoRAG** [4] draws on associative-retrieval neuroscience for multi-hop QA over a graph index. These systems are valuable *memory layers*: they orchestrate extraction, summarization, and retrieval over general-purpose backends. We position FluctlightDB as the missing *engine*

beneath such layers—where the native operations are `experience` and `activate`, not `SQL` or `vector_search`.

7.3 Positioning

To our knowledge, no prior work argues that long-term agent memory is a *candidate* third data model—peer to the relational [2] and vector models—with engine-level write and read semantics in an embedded store. Enterprise databases adding agent features (e.g. Oracle AI Database unified memory) retrofit memory onto existing engines rather than defining episodes and cues as first-class types. Table 7 summarizes the landscape; direct head-to-head retrieval on LoCoMo evidence recall against Mem0/Zep is future work (different published metrics today).

Table 7: Landscape: memory *layers* vs. FluctlightDB as an embedded *engine*. **Top block:** our internally reproduced metrics. **Bottom block:** cited vendor/literature figures on *different* metric types—not head-to-head.

System	Kind	Native con- tract	Benchmark	Metric type
<i>FluctlightDB (this work; internally reproduced)</i>				
FluctlightDB	Engine	<code>experience</code> / <code>activate</code>	99.0%	LoCoMo evidence
FluctlightDB	Engine	hybrid index + session keys	97.6%	LongMemEval session@8
FluctlightDB	Engine	Muon + paper E2E	97.4%	LongMemEval E2E QA
<i>Cited vendor/literature (different metrics; not comparable)</i>				
Mem0 Mem0 ^s	/ SDK + backend	Extract / consolidate / retrieve facts	~92%+ LLM-J	End-to-end QA
Zep	Managed layer	Temporal KG + summaries	~75% LLM-J	End-to-end QA
Cognee	Pipeline	Graph + vector ETL	—	Task-specific
MemGPT Letta	/ Agent OS	Context tiers / blocks	—	Session QA
HippoRAG	Graph RAG	Associative graph retrieval	—	Multi-hop QA
Chroma Qdrant	/ Vector DB	ANN top- <i>k</i>	—	IR / similarity

Benchmarks LoCoMo [8] and LongMemEval [14] measure long-horizon memory; BEIR [12] anchors IR credibility. Our framing—memory as a *proposed* data model on equal footing with rows and vectors—is the argument this engine and its measurements are built to test.

8 Conclusion

The relational model gave applications a database for *facts*; the vector model gave search a database for *similarity*. Autonomous agents need a database for *memory*, and it should be as boring to adopt and as rigorous to trust as SQLite. FluctlightDB is our argument that this engine can

exist today: it ties vector baselines where they are strong on shared harnesses, passes our author-designed agent-memory regression suites at 100% macro, recalls 99.0% of gold evidence on LoCoMo retrieval, 97.6% session recall on LongMemEval-S (retrieval v4), and 97.4% end-to-end QA on LongMemEval-S—while LoCoMo end-to-end QA remains weak (23.5% F1 on a 50-question pilot). We release everything needed to reproduce and contest these claims.

Artifacts

Repository: FluctlightDB (MIT). Harnesses: `benchmarks/locomo_eval.py`, `benchmarks/longmemeval_bench.py`, `benchmarks/longmemeval_colab.ipynb`, `benchmarks/beir_bench.py`, `benchmarks/agent_memory_bench.py`. Frozen metrics: `benchmarks/results/paper-2026-07-09.json`. E2E: `benchmarks/longmemeval_e2e.py`; `benchmarks/results/e2e-cert-paper-v2-2026-07-07.json`. Preprint DOI: <https://doi.org/10.5281/zenodo.20949890>. Source: <https://github.com/voxmastery/FluctlightDB/tree/main/papers/arxiv-v1>.

References

- [1] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [2] Edgar F Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [3] Ila R. Fiete, Yoram Burak, and Ted Brookings. What grid cells convey about rat location. *Journal of Neuroscience*, 28(27):6858–6871, 2008.
- [4] Bernardo Gutiérrez, Yiheng Shu, Yu Gu, Peter Pasupat, Barret Liu, Rishabh Shah, Ming-Wei Chang, Ramakanth Pasunuru, Yujia Zhang, Avi Gupta, et al. Hipporag: Neurobiologically inspired long-term memory for large language models. *arXiv preprint arXiv:2405.14831*, 2024.
- [5] Richard D. Hipp. SQLite. <https://www.sqlite.org/>, 2024. Embedded relational database engine.
- [6] John E. Lisman and Marco A. P. Idiart. Storage of 7 ± 2 short-term memories in oscillatory subcycles. *Science*, 267(5203):1512–1515, 1995.
- [7] John E. Lisman and Ole Jensen. The theta-gamma neural code. *Neuron*, 77(6):1002–1016, 2013.
- [8] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *ACL*, 2024.
- [9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [10] Tony A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.

- [11] Sameet Sreenivasan and Ila Fiete. Grid cells generate an analog error-correcting code for singularly precise neural computation. *Nature Neuroscience*, 14(10):1330–1337, 2011.
- [12] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *NeurIPS Datasets and Benchmarks*, 2021.
- [13] Topoteretes. Cogne: Memory engine for ai agents. <https://github.com/topoteretes/cogne>, 2024. Vector + knowledge-graph memory pipeline.
- [14] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: A benchmark for long-term memory in llm chat assistants. *arXiv preprint arXiv:2410.10813*, 2024.
- [15] Zep AI. Zep: A temporal knowledge graph architecture for agent memory. <https://www.getzep.com>, 2024. Agent memory platform; LoCoMo evaluations in product documentation.